FLIPKART

E-COMMERCE PLATFORM CLONE

High-Level Design Document | Version 1.0

Field	Details
Document Title	Flipkart Clone — High-Level System Design
Document Type	High-Level Design (HLD)
Version	
Author	Aryan Verma
Date	April 2026
Tech Stack	React.js · Node.js · Express.js · PostgreSQL · Prisma ORM
Architecture	Modular Monolith — Single Backend, Single Frontend
Deployment Target	Vercel (Frontend) · Render (Backend) · Supabase (Database)

Table of Contents

1. Solution Overview

> *This document describes the High-Level Design of a Flipkart-clone e-commerce web application built as an SDE Intern Assignment. It covers architecture, components, data flows, use cases, and design decisions at a conceptual level, without implementation-level code detail.*

The Flipkart Clone is a full-stack, production-ready e-commerce web application that replicates the core shopping experience of Flipkart. It enables users to browse a product catalogue, manage a shopping cart, place orders, maintain a wishlist, and view their order history — all within a pixel-accurate recreation of Flipkart's user interface.

The system is designed as a Modular Monolith: a single cohesive backend application internally structured around well-separated domain modules (Products, Cart, Orders, Auth, Wishlist), and a single-page React frontend. This architectural choice deliberately optimises for developer velocity, evaluator readability, and deployment simplicity — the correct trade-off for a two-day intern assignment scope while still demonstrating production-grade engineering principles.

1.1 Primary System Components

Component	Technology	Responsibility
Frontend SPA	React.js (Vite)	User interface, routing, client-side state
Backend API	Node.js + Express.js	Business logic, REST API, authentication
Database	PostgreSQL (Supabase)	Persistent storage for all domain entities
ORM	Prisma	Type-safe DB queries, schema management, migrations
Auth Tokens	JWT (httpOnly Cookies)	Stateless auth with server-side refresh token revocation
State Management	Zustand	Client-side cart, auth, and wishlist state
Deployment	Vercel + Render	Frontend CDN + Backend web service

1.2 Out of Scope

The following items are explicitly out of scope for this version:

- Admin dashboard and product management UIo
- Product image upload (CDN / S3)
- Real-time inventory sync
- Search engine integration (Elasticsearch / Algolia)
- Microservices decomposition

2. Requirements Reference

2.1 Functional Requirements

2.1.1 Core Features (Must Have)

ID	Feature	Description
FR-01	Product Listing	Display products in a responsive grid with search by name and filter by category
FR-02	Product Detail	Image carousel, specifications, price/MRP/discount, stock status, Add to Cart, Buy Now
FR-03	Shopping Cart	Add/update/remove items, quantity stepper, price breakdown with savings summary
FR-04	Order Placement	Checkout with address form, order summary review, order confirmation with ID

2.1.2 Bonus Features

ID	Feature	Description
FR-05	Responsive Design	Fully functional across mobile (390px), tablet (768px), and desktop (1440px)
FR-06	User Authentication	JWT-based Login/Signup with access + refresh tokens in httpOnly cookies
FR-07	Order History	Logged-in users can view all past orders with status and item summary
FR-08	Wishlist	DB-persisted wishlist tied to user; toggle from product card and detail page

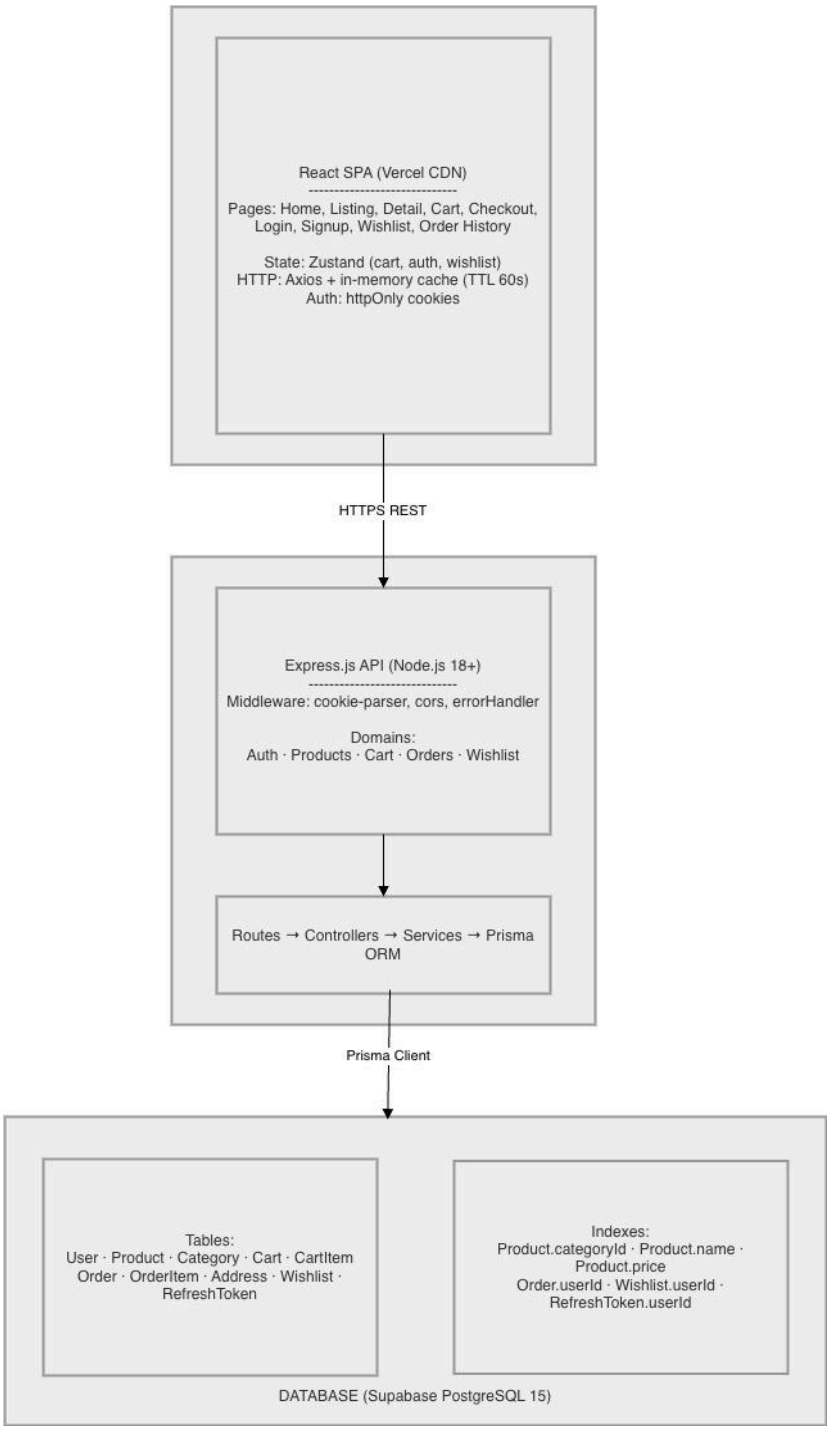
2.2 Non-Functional Requirements

Category	Requirement	Target
Performance	API response time (p95)	< 500ms for product listing
Performance	Frontend initial load	< 3s on 4G connection
Availability	Backend uptime	> 99% (Render free tier — cold start caveat noted)
Security	Password storage	bcrypt with 12 salt rounds

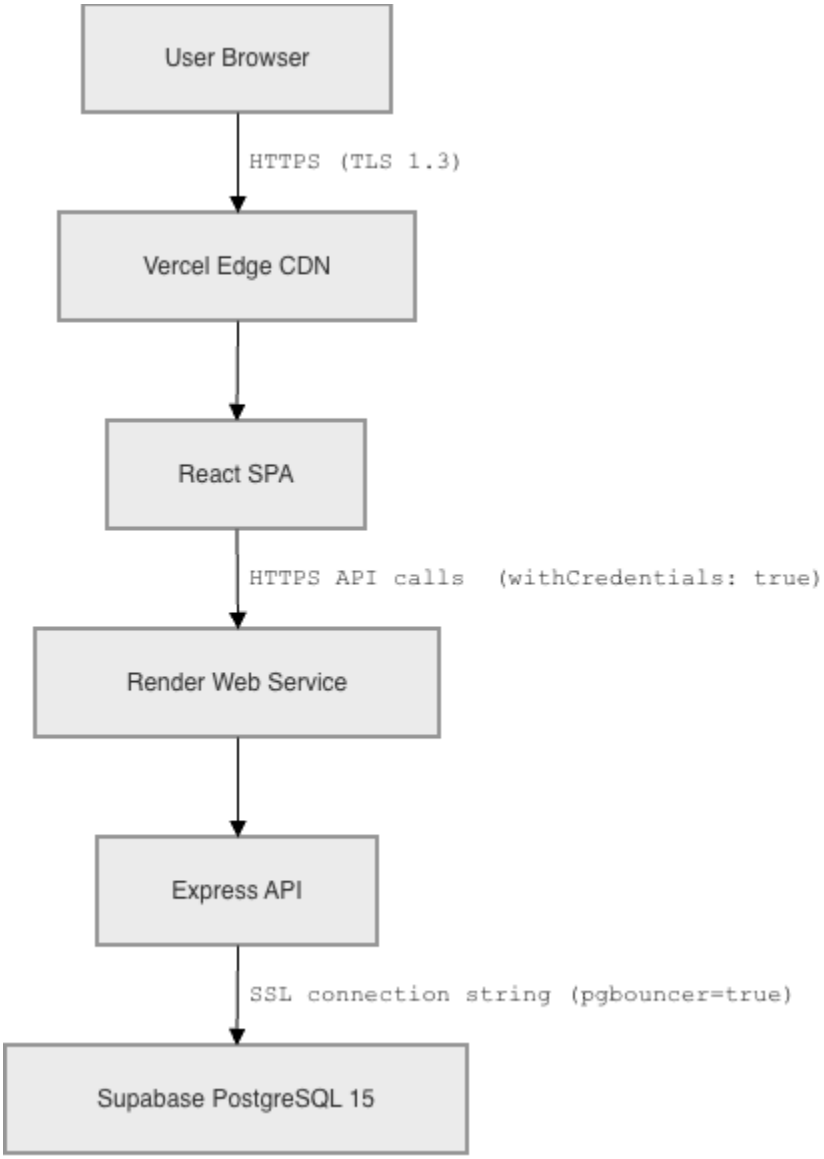
Category	Requirement	Target
Security	Token storage	httpOnly, sameSite: strict cookies — no localStorage
Security	CORS policy	Explicit origin whitelist, no wildcard
Scalability	DB query strategy	No N+1 queries; indexed columns on all filter fields
Usability	Mobile touch targets	Minimum 44px per Apple HIG guidelines
Reliability	Cart persistence	Cart survives page refresh via server-side storage
Reliability	Order atomicity	Stock decrement and order creation in a single DB transaction
Maintainability	Code organisation	Strict layer separation: routes / controllers / services / DB
Observability	Error handling	Global error handler with consistent JSON error shape

3. High-Level Architecture Diagram

The system follows a classic three-tier architecture: Presentation (React SPA), Application (Express API), and Data (PostgreSQL). The tiers are deployed independently on separate cloud services and communicate over HTTPS.



3.1 Deployment Topology



4. Logical Breakdown — Module Decomposition

Although the system is deployed as a Modular Monolith, it is internally structured around five distinct domain modules. Each module owns its own routes, controller, service, and Prisma queries — enabling independent development, testing, and future extraction into microservices if scale demands it.

4.1 Backend Module Map

Module	Routes	Key Responsibilities	External Dependencies
Auth	POST /auth/signupPOST /auth/loginPOST /auth/refreshPOST /auth/logoutGET /auth/me	User registration, bcrypt hashing, JWT issuance, refresh token rotation, guest cart merge on login	bcrypt · jsonwebtoken · crypto
Products	GET /productsGET /products/:idGET /categories	Paginated product listing with filters (search, category, price, sort), single product fetch, category list	Prisma full-text search
Cart	POST /cartGET /cart/:idPOST /cart/:id/itemsPATCH /cart/:id/items/:pidDELETE /cart/:id/items/:pid	Guest and user cart lifecycle, upsert on add, quantity update, cart-to-user migration on login	Prisma upsert
Orders	POST /ordersGET /ordersGET /orders/:id	Atomic order placement (Prisma transaction), stock decrement, address storage, order history by userId	Prisma $transaction
Wishlist	GET /wishlistPOST /wishlist/:pidDELETE /wishlist/:pid	User wishlist CRUD, toggle endpoint, batch productId Set for frontend	authenticate middleware

4.2 Frontend Module Map

Layer	Files	Responsibility
Pages	HomePage · ProductListingPage · ProductDetailPage · CartPage · CheckoutPage · OrderConfirmationPage · LoginPage · SignupPage · WishlistPage · OrderHistoryPage	Route-level components; orchestrate hooks and layout
Components	Navbar · CategoryBar · ProductCard · ProductGrid · ProductCardSkeleton · ImageCarousel · RatingBadge · SpecsTable ·	Presentational; receive props, emit callbacks, no direct API calls

Layer	Files	Responsibility
	FilterSidebar · SortDropdown · CartItem · PriceSummaryPanel · AddressForm · OrderSummaryPanel · WishlistButton · ProtectedRoute · EmptyState · Spinner	
Hooks	useProducts · useProduct · useCart · useOrder · useOrderHistory	Data fetching; wrap API calls, manage loading/error state
Stores	cartStore · authStore · wishlistStore	Zustand stores; global client state with persistence where needed
API Layer	apiClient · productApi · cartApi · orderApi · authApi · wishlistApi	Axios wrappers; single source for all API calls, in-memory cache, 401 refresh interceptor
Utils	formatCurrency · calculateDiscount · debounce	Pure functions; reusable across components and hooks

4.3 New Modules / Skills Required (Bonus Features)

Module	New or Modified?	New Skills / Libraries
Auth Module	New	bcrypt, jsonwebtoken, cookie-parser, crypto (Node built-in)
Wishlist Module	New	No new libraries — uses existing Prisma + authenticate middleware
Order History	Modified	New GET /orders route added to existing Orders module
Responsive CSS	Modified	CSS media queries on all existing components — no new library
Cart-User Migration	Modified	mergeGuestCart() added to authService — no new library
Token Refresh	New	Axios response interceptor with queue for concurrent 401 handling

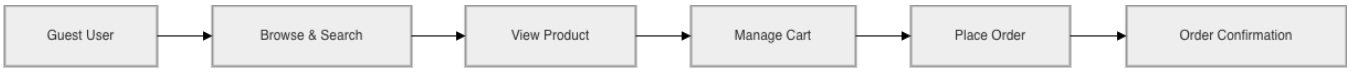
5. Business Processes, Use Cases, and User Stories

5.1 Actors

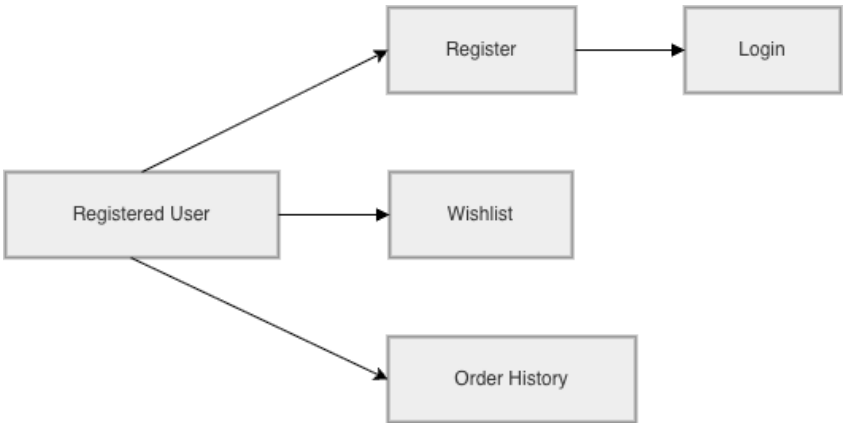
Actor	Description
Guest User	Unauthenticated visitor; can browse, search, filter, view product details, and manage a cart. Cannot access wishlist or order history.
Registered User	Authenticated user; all guest capabilities plus wishlist, order history, and cart persistence tied to their account.
System	The backend — enforces business rules such as stock validation, order atomicity, and token refresh.

5.2 Core Use Cases

```
┌──────────────────────────────────────────────────────────────────────┐
│                        SYSTEM BOUNDARY                               │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  UC-01  Browse and Search Products                           │   │
│   │  UC-02  View Product Detail                                  │   │
│   │  UC-03  Manage Shopping Cart                                 │   │
│   │  UC-04  Place an Order                                       │   │
│   │  UC-05  View Order Confirmation                              │   │
│   │  UC-06  Register an Account                                  │   │
│   │  UC-07  Login / Logout                                       │   │
│   │  UC-08  Manage Wishlist                                      │   │
│   │  UC-09  View Order History                                   │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  Actors:  Guest User ──────────────────────────────── UC-01 to 05    │
│           Registered User ──────────────────────────── UC-01 to 09   │
└──────────────────────────────────────────────────────────────────────┘
```



Guest flow Diagram



Registered User Flow Diagram

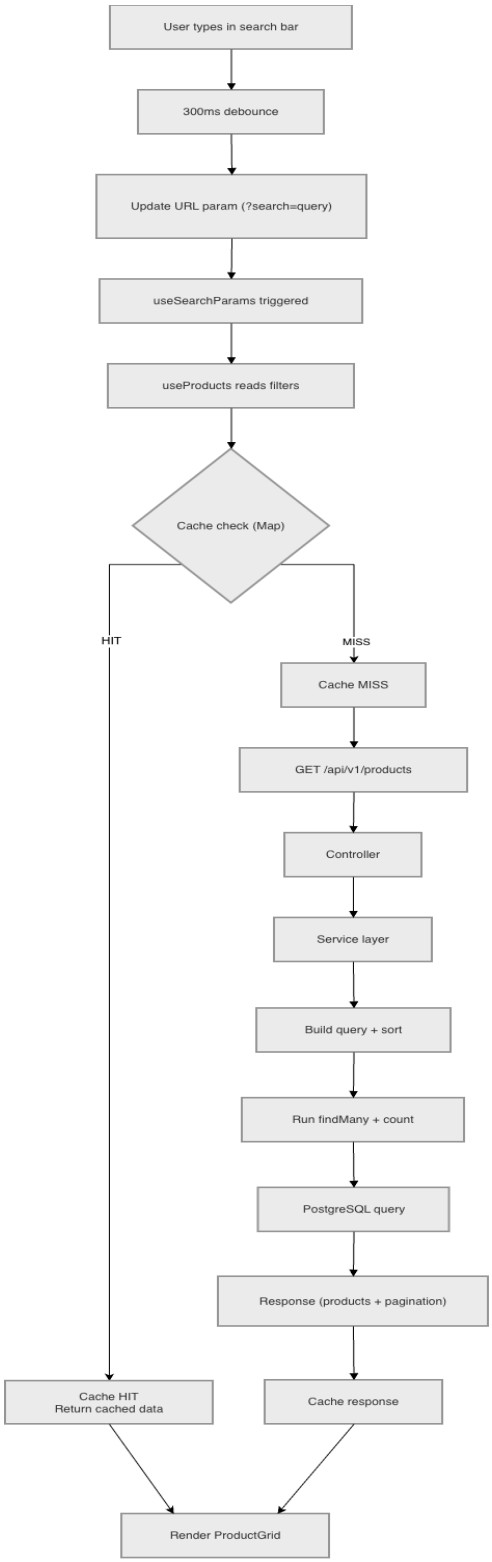
5.3 User Story Map

ID	As a…	I want to…	So that…	Acceptance Criteria
US-01	Guest User	Search and filter products	I can quickly find what I want	Results update within 300ms of typing; category and price filters refine results in a single API call
US-02	Guest User	View product detail with images	I can make an informed purchase decision	Image carousel shows all product photos; specs table renders from JSON; stock status shown clearly
US-03	Guest User	Add items to cart	I can collect items before purchasing	Cart persists on page refresh via server storage; Navbar count updates immediately
US-04	Guest User	Update cart quantities	I can buy the right amount	Optimistic update feels instant; reverts cleanly on server error
US-05	Guest User	Place an order	I can complete a purchase	Order is atomic; stock cannot go negative; confirmation page shows order ID
US-06	Guest User	Register an account	I can access personalised features	Password hashed with bcrypt; tokens set as httpOnly cookies; no plain-text credentials in transit
US-07	Registered User	Login with email and password	I can access my wishlist and history	Guest cart is merged into user cart on login; redirect returns to original destination

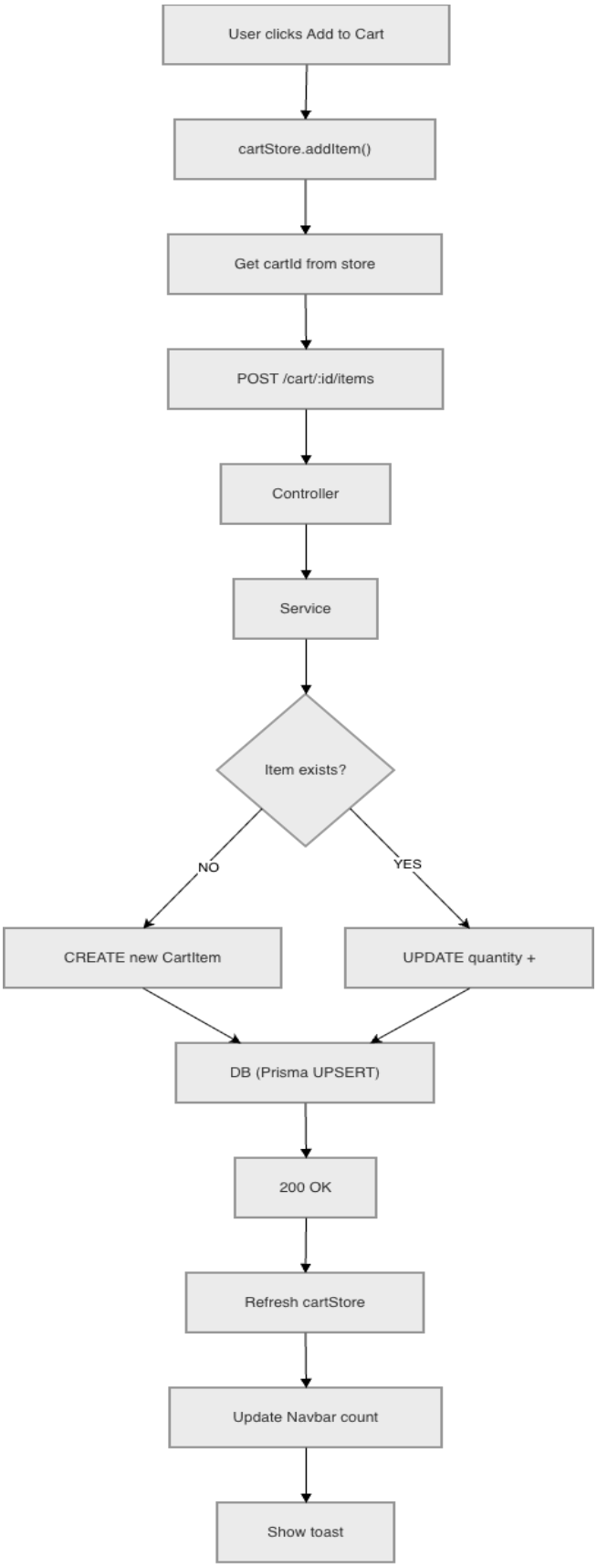
ID	As a…	I want to…	So that…	Acceptance Criteria
US-08	Registered User	Add products to my wishlist	I can save items I like for later	Heart icon fills immediately (optimistic); wishlist persists across sessions in DB
US-09	Registered User	View all past orders	I can track my purchase history	All orders shown newest-first; each shows status badge, item thumbnails, total, and delivery address

6. Data Flow Diagrams

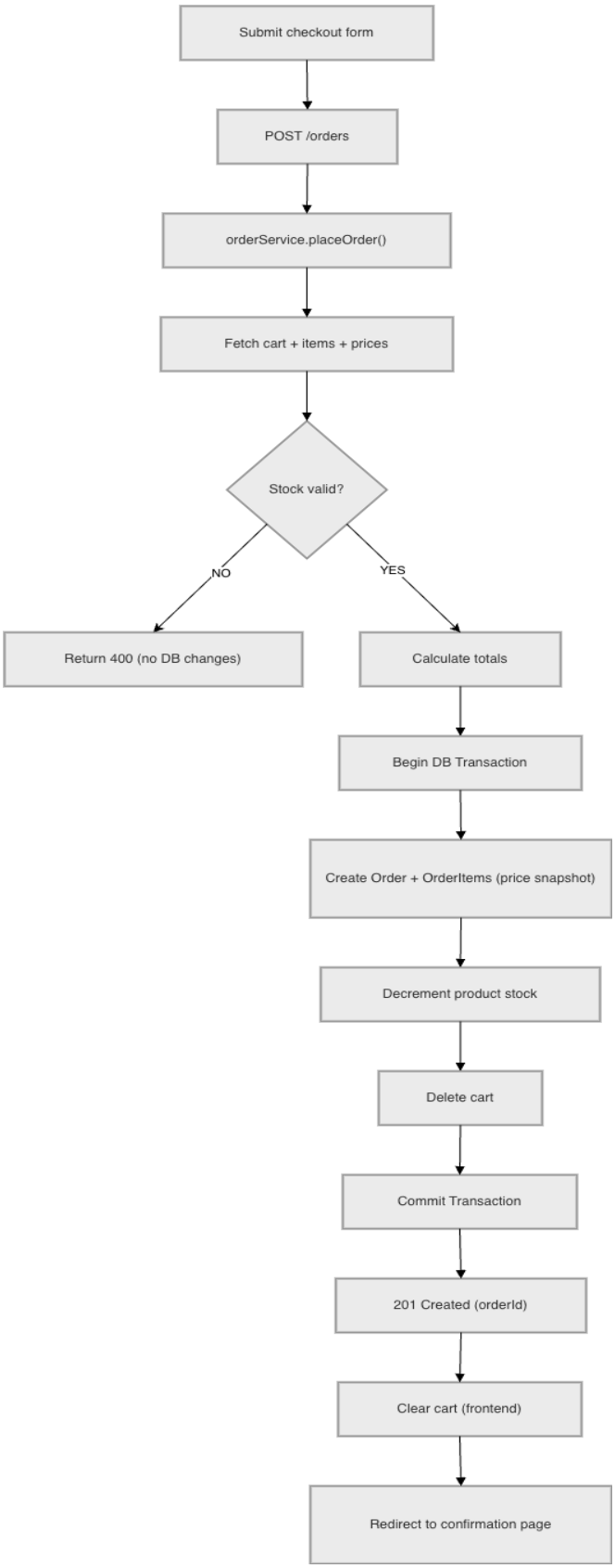
6.1 Product Search and Listing Flow



6.2 Add to Cart Flow (Optimised)

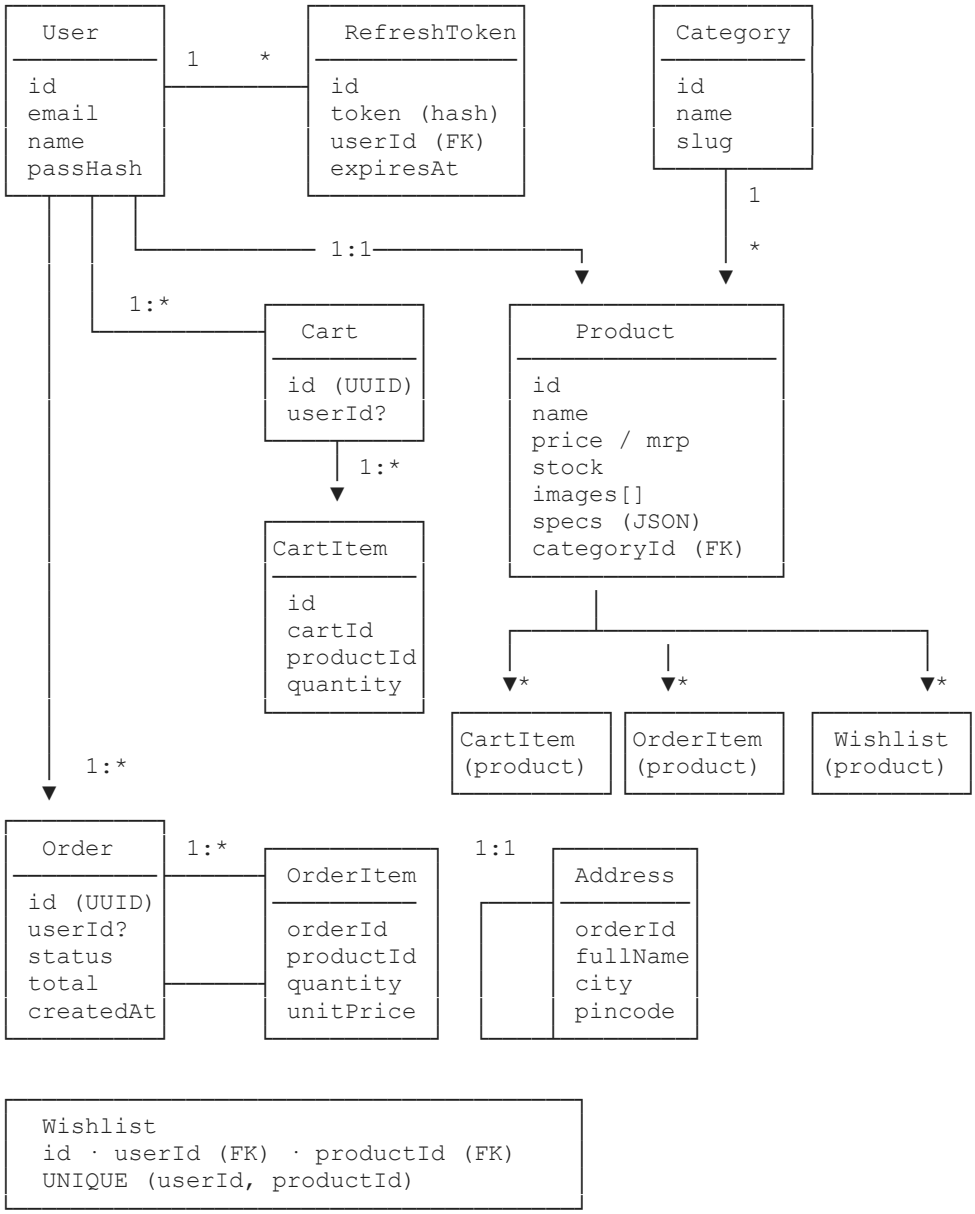


6.3 Order Placement Flow (Atomic Transaction)

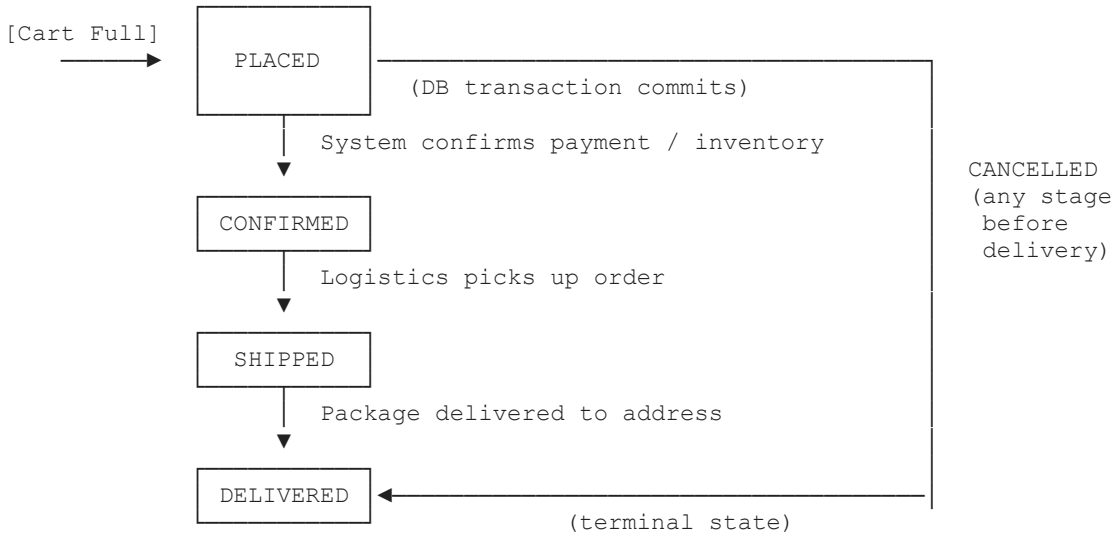


7. Other Representations

7.1 Entity Relationship Diagram

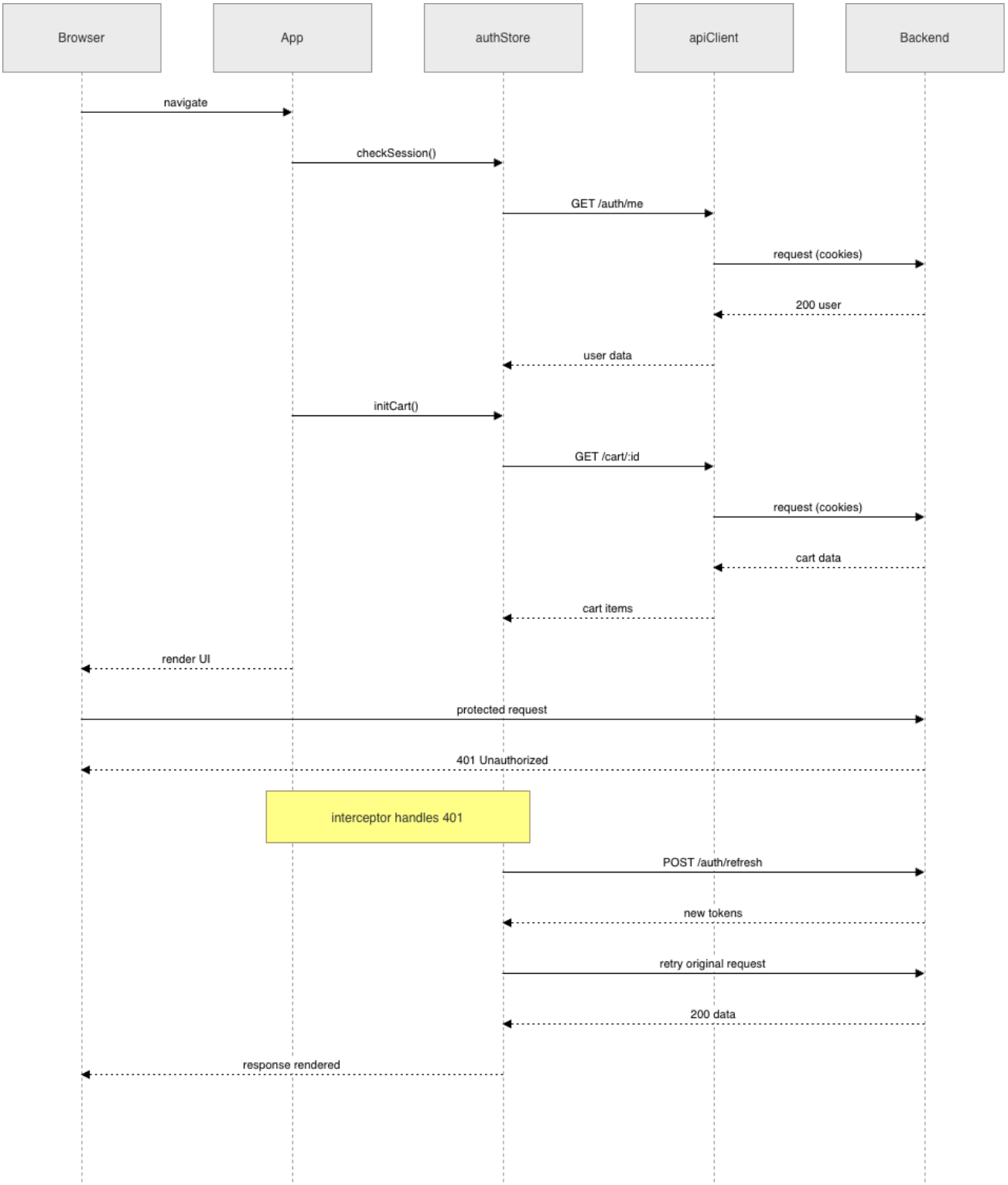


7.2 State Machine — Order Lifecycle



Note: Status transitions are enum-controlled in DB.
CANCELLED is a terminal state accessible from PLACED, CONFIRMED, or SHIPPED.

7.3 Temporal Sequence — Page Load and Session Check



7.4 Component Interaction — Cart State

```
┌──────────────────────────────────────────────────────────────────────┐
│  ZUSTAND cartStore                                                   │
│  { cartId, items[], loading }                                        │
│  Derived: itemCount, subtotal, savings                               │
└──────────────────────────────────────────────────────────────────────┘
        │  reads                                  │  writes
        ▼                                         ▼
┌──────────────────┐                    ┌──────────────────────┐
│ Navbar.jsx       │                    │ ProductDetailPage    │
│ cart count       │                    │ AddToCartButton      │
│ badge            │                    │ BuyNowButton         │
└──────────────────┘                    └──────────────────────┘
                                                  │ addItem()
                                                  ▼
┌──────────────────┐                    ┌──────────────────────┐
│ CartPage         │                    │ cartApi.addItem()    │
│ CartItem ×n      │  updateQty()       │ POST /cart/:id/items │
│ PriceSummary     ├──────────────────► │ Prisma UPSERT        │
└──────────────────┘  removeItem()      └──────────────────────┘
        │  reads                                  │  response
        ▼                                         ▼
Optimistic update applied to local state immediately
Server confirms → state stays    |    Server error → revert
```

8. Security Design

Threat	Mitigation	Where Applied
XSS (token theft)	Tokens stored in httpOnly cookies — inaccessible to JavaScript	cookie-parser + res.cookie() with httpOnly: true
CSRF	sameSite: strict on all cookies — browser blocks cross-origin sends	All auth cookies
Password breach	bcrypt with cost factor 12 — deliberately slow hash	authService.signup() and login()
User enumeration	Identical error for wrong email and wrong password	authService.login()
Refresh token replay	Token rotation — old token deleted on use; stolen token fails next use	authService.refreshTokens()
DB refresh token breach	Only SHA-256 hash stored in DB — raw token only lives on client	createRefreshToken() uses crypto.createHash('sha256')
Overselling (race condition)	Stock validation + order creation in single DB transaction	prisma.$transaction() in orderService
Cross-origin API abuse	CORS restricted to explicit Vercel domain — no wildcard	server.js cors({ origin: [FRONTEND_URL] })
Unauthorised data access	authenticate middleware on all user-specific routes	Wishlist, Order History, GET /auth/me
Input injection	Zod schema validation on all request bodies before reaching service	validate.js middleware on all POST/PATCH routes

9. API Summary

All endpoints follow the standard response envelope:

```
Success: { "success": true,  "data": {},    "message": "optional" }
Error:   { "success": false, "error": "Human readable message"    }
```

Method	Endpoint	Auth Required	Description
GET	/products	No	Paginated product list — accepts search, category, minPrice, maxPrice, sort, page, limit
GET	/products/:id	No	Single product with category
GET	/categories	No	All categories for the category bar
POST	/cart	No	Create new cart — returns { cartId }
GET	/cart/:cartId	No	Cart with all items and product details
POST	/cart/:cartId/items	No	Add or increment item { productId, quantity }
PATCH	/cart/:cartId/items/:productId	No	Update item quantity
DELETE	/cart/:cartId/items/:productId	No	Remove item from cart
POST	/orders	No*	Place order — { cartId, address }. Attaches userId if cookie present
GET	/orders	Yes	Order history for the authenticated user
GET	/orders/:orderId	No	Single order detail with items and address
POST	/auth/signup	No	Register — { name, email, password }
POST	/auth/login	No	Login — { email, password, guestCartId? }
POST	/auth/refresh	No	Rotate refresh token — reads cookie automatically
POST	/auth/logout	No	Revoke refresh token and clear cookies
GET	/auth/me	Yes	Return current user from access token
GET	/wishlist	Yes	All wishlist items with product details
POST	/wishlist/:productId	Yes	Add product to wishlist
DELETE	/wishlist/:productId	Yes	Remove product from wishlist

10. Performance and Optimisation Strategy

Layer	Optimisation	Impact
Frontend	300ms debounce on search input	Eliminates keystroke-per-request — reduces API calls by ~90% during typing
Frontend	All filters bundled in one GET request	One round-trip regardless of how many filters are active simultaneously
Frontend	In-memory Axios cache (Map, TTL 60s)	Back-navigation from detail to listing is instant — zero additional requests
Frontend	Optimistic UI on cart mutations	Quantity updates and removes feel instantaneous; only revert on server error
Frontend	Zustand cart count derived from store	Navbar badge updates without an extra API call on every action
Frontend	Wishlist productId Set for O(1) lookup	Heart icon fill state on listing page requires no per-card API calls
Frontend	loading='lazy' on all product images	Below-fold images load on demand, reducing initial page weight
Backend	Two parallel Prisma queries (findMany + count)	Product count and page data fetched concurrently, halving query round-trips
Backend	Nested Prisma includes eliminate N+1 queries	Cart, orders, wishlist all loaded in a single DB round-trip with JOINs
Database	B-tree indexes on categoryId, name, price	Category filter, search, and price range queries hit indexes — O(log n) vs O(n)
Database	Composite index on Order.userId	Order history queries are O(log n) regardless of total order volume
Database	pgbouncer connection pooling (Supabase default)	Prevents connection exhaustion under concurrent load

11.Design Decisions and Tradeoffs

Decision	Chosen Approach	Alternative	Reasoning
Architecture	Modular Monolith	Microservices	2-day timeline; equal internal modularity; no Docker/networking overhead; easy for evaluator to read and run
Token Storage	httpOnly cookies	localStorage JWT	httpOnly cookies are immune to XSS; sameSite:strict mitigates CSRF — production-correct approach
Product Specs	JSON column	EAV relational table	Specs are heterogeneous per category; EAV requires 3-way JOINs; JSON is read-heavy friendly
Product Images	String[] array column	ProductImage table	Ordered URL list; no JOIN needed; Postgres native arrays are perfect for this use case
Cart Persistence	Server-side (DB) + UUID cookie	localStorage only	Server cart survives clearing browser storage; enables cart merge on login
Order Price	Snapshot unitPrice on OrderItem	Live product price JOIN	Product prices can change; historical orders must show what was paid — standard e-commerce practice
Filter State	URL search params	Component state	Back button works; filters survive page refresh; URLs are shareable and bookmarkable
ORM	Prisma	Raw SQL / Sequelize	Type-safe queries; readable schema.prisma is self-documenting for evaluators; migration history is clean
Frontend State	Zustand	Redux / Context	Minimal boilerplate; no providers; computed values; easy to read in an interview setting
Wishlist Lookup	Set<productId> in store	Per-card API call	O(1) lookup; single fetch on mount; zero per-card API overhead on listing page with 54 products

12. Assumptions and Known Limitations

12.1 Assumptions

- A default guest user flow is supported — no login is required to browse or purchase.
- All product images are served from external URLs (Picsum placeholders in development); no image upload is implemented.
- Delivery is free for all orders; no shipping cost calculation is applied.
- A single INR currency is used throughout; no multi-currency support.
- The database is pre-seeded with 54 sample products across 6 categories before first use.
- Email / SMS notifications are out of scope; order confirmation is shown in-browser only.
- Payment processing is out of scope; orders are placed without a payment gateway.

12.2 Known Limitations

Limitation	Impact	Future Resolution
Render free tier cold start (~30s)	First API request after 15min idle is slow	Upgrade to paid tier or add a keep-alive ping service
In-memory cache is per-tab	Cache not shared across browser tabs	Use a service worker or BroadcastChannel API for cross-tab cache
No real-time stock updates	Stock shown at page load time only	Add WebSocket or SSE for live inventory updates
No payment gateway	Orders are placed without actual payment	Integrate Razorpay or Stripe in v2
Product search is ILIKE only	No fuzzy search or relevance ranking	Replace with PostgreSQL full-text search or Elasticsearch
Refresh token stored as single row	No multi-device session management	Add deviceId to RefreshToken model for per-device logout
No rate limiting on auth routes	Brute-force login attempts possible	Add express-rate-limit to /auth/login and /auth/signup